Vector Semantics and Embeddings

Dr. Satadisha Saha Bhowmick

Preceptor

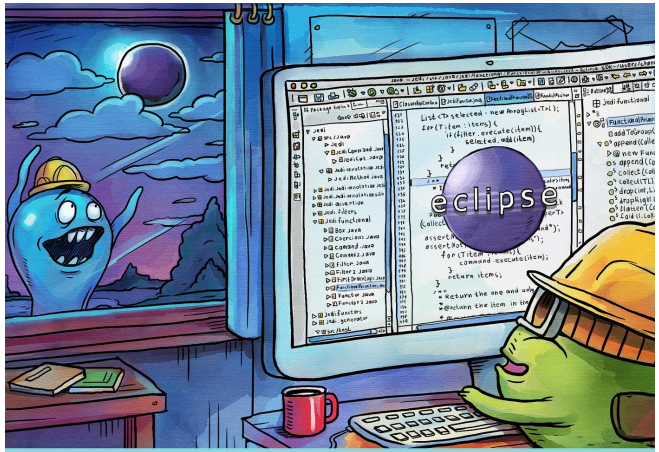
THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Objectives

- Motivation
- Lexical Semantics
- Vector Semantics
- Cosine for Word Similarity
- TF-IDF Weighting Factor
- Word2Vec
- Semantic Properties of Embeddings
- Bias and Embeddings
- Evaluation

Motivation

- ***Distributional Semantics*** – A word's meaning is given by the words that frequently accompany it within text.
- This link between similarity in how words are distributed and similarity in what they mean is called the ***distributional hypothesis***.
  Hypothesis first formulated in 1950s.
- A *localist representation* of words is therefore to produce a vectorial representation in the context in which they appear in text.
  Variable context can draw variable meanings for the same word.
- ***Vector Semantics*** involve processes to learn these representations of words while abiding by distributional hypothesis. Represents a word in multidimensional space as features like most ML problems.
  - Static Representations or embeddings do not change with the context in which a word appears.
  - Contextualized embeddings vary everytime the word appears in a new context, surrounded by new set of words.
- Representation Learning is an important foundational task for any NLP system.



- Look! The eclipse!
- So what? I use it everyday to write my Java code.
- Here! In the window!
- Well, obviously not in the console!

Lexical Semantics

Some basic principles and terminologies surrounding the ***Lexical Semantics*** -- linguistic study of word meaning-- that we use in NLP

- **Lemma, Wordforms and Senses** : The word mouse could be defined in the dictionary in more than one ways:
  1. small rodent
  2. a hand-operated device that controls a cursor

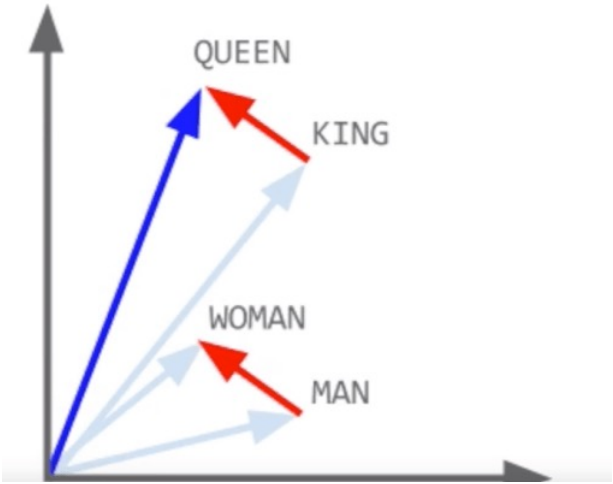
  The (citation) form or *wordform* is pretty much how a word appears as is in text however the root of wordforms is called *lemma*. Wordforms mouse or mice both have lemma mouse. However, mouse can have either of the aforementioned senses depending on the context.
- **Synonymy** : Relationship between senses of two different words. If these two senses are equivalent that is an evidence of synonymy. In practice, the word *synonym* is therefore used to describe a relationship of approximate or rough synonymy.
  couch/sofa; *car/automobile*
- One of the fundamental tenets of semantics, called the **Principle of Contrast** states that difference in lemma is always associated with some difference in meaning. For example, the word *H2O* is used in scientific contexts but could be less inappropriate in a hiking guide— instead of *water*— and this genre difference is part of the meaning of the word.

Lexical Semantics

Some basic principles and terminologies surrounding the ***Lexical Semantics*** -- linguistic study of word meaning-- that we use in NLP

- **Word Similarity**: Words have few synonyms but lots of similar words. This notion of word similarity is useful in larger semantic tasks: Question Answering, Paraphrasing, Summarization etc. Moving from exact synonymy to inexact word similarity means semantic tasks can shift from focus relation between word senses to words.
  Example: For many NLP tasks, *cat* is not a synonym of *dog*, but *cats* and *dogs* are certainly similar words.
- **Word Relatedness**: The meaning of two words can be related in ways other than similarity. One such class of connections is called word relatedness.
  Example: *coffee* is related to *cup* and appear in similar contextual settings despite not being synonyms.
- **Semantic Field**: Refers to a set of words covering a particular semantic domain.
  Example: the semantic field of hospitals (*surgeon, scalpel, nurse, anesthetic, hospital*), restaurants (*waiter, menu, plate, food, chef*), or houses (*door, roof, kitchen, family, bed*).
- **Topic Models**: Topic modeling is a type of statistical modeling that uses unsupervised Machine Learning to identify clusters or groups of similar words within a body of text that correspond to a broad topic or semantic concept.

Vector Semantics



- The study of representing the meaning of a word for NLP modeling.
- A common line of thought in this area is to define the meaning of a word (esp through its vector representation) by its **distribution** in language use (defined by its neighboring words- lexical and grammatical environments). *A word is defined by the company it keeps!*

Example: Borrowing from Cantonese, let's assume that we didn't know the meaning of the word ong-choy but we find it appearing in the following contexts:

- *Ong-choy is delicious when sauteed with garlic.*
- *Ong-choy is great with rice.*

vs another word that appears within the same context words

- *Spinach is delicious when sauteed with garlic.*
- *Spinach is great with rice.*

- We represent a word as a point in a ***multidimensional semantic space*** that is derived from the distributions of word neighbors.
  Vectors that are trained to represent words are called ***embeddings***.
- With vector representations you can extend all the concepts of vectors and linear algebra to the study of vector semantics.
  - Project similarity between words through the similarity between their embedding vectors.

Words and Vectors

Distributional representations for words are generated by leveraging the context words around it. We start by building a ***co-occurrence matrix*** to indicate how often words co-occur.

- We want to represent words and documents as vectors in a multidimensional space called the ***vector space***.
  Word vectors formed through their occurrence in documents while document vectors are distribution of words.
  Two popular matrices are necessary for a corpus.
- Term Document Matrix:
  - Each row is a word in the vocabulary and each document is a column from the corpus. What we record are raw counts of a specific word in a specific document.
  - Consists of $|V|$ rows which is the size of the vocabulary set and $|D|$ columns which is the corpus size. Each document is represented by its column vector of size $|V|$. Each word is represented by its row vector of size $|D|$.
- Term Term Matrix:
  - Each row and each column represents a word in the vocabulary and the matrix dimensionality is $|V|\times|V|$.
  - Cell (i, j) record the number of times $word_i$ and $word_j$ co-occur.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Co-occurrence vectors for four words in the Wikipedia corpus.

Similar documents and similar words are expected to have similar vectors and reflect that spatially as well when these vectors are visualized.

Typically, $|V|$ the dimensionality is in the scale of 100,000

Cosine for measuring similarity

Similarity between two vectors is typically given by the cosine of the angle between them or the normalized dot product – ***Cosine Similarity***
To measure similarity between two target words or documents in the vector space model we use the cosine similarity.

- Dot product is a linear algebra operation also called inner product
- Dot product acts as a similarity metric because it tends to have a high value when two vectors have similar values along the same dimensions.
- Dot product will be 0 when two vectors are '*orthogonal*' when vectors have zeros in different dimensions – represents strong dissimilarity!
- Unnormalized dot product as a similarity metric favors longer vectors with a greater number of dimensions. Hence, we go for normalized dot product, where the sum of dimensional products are normalized by vector lengths.

  Length of a vector v : $|v| = \sqrt{\sum_{i=1}^{N} v_i^2}$
- For some applications we pre-normalize each vector, by dividing it by its length, creating a unit vector of length 1. Cosine similarity is normalized to the range $(-1,1)$. 1 for vectors that point in the same direction, 0 for orthogonal vectors and -1 for vectors in the opposite direction.
  In the vector space model, since raw frequencies are positive, the range is 0-1.

$$dot-product(v,w) = v \cdot w$$
$$= v_1 w_1 + v_2 w_2 + \cdots + v_n w_n = \sum_{i=1}^{n} v_i w_i$$
$$v \cdot w = |v||w| \cos\theta$$
$$\cos\theta = \frac{v \cdot w}{|v||w|}$$
$$cosine\ (v,w) = \frac{\sum_{i=1}^{n} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

Let's see how the cosine computes which of the words *cherry* or *digital* is closer in meaning to *information*, just using raw counts from the following shortened table:

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry, information}) = \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .018$$
$$\cos(\text{digital, information}) = \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$

TF-IDF Weighting Factor

- Raw frequency is not the best measure of association between words. Raw frequency is very skewed and not very discriminative.
- Some words will always appear too many times in most corpora and are not informative. Words like *the*, *it*, or *they* (**Stopwords**). Or in Shakespearean corpus the dimension of the word *good* is not very discriminative amongst plays– just a frequent word!
- A paradox : words that occur nearby frequently (maybe *pie* nearby *cherry*) are more important than words that only appear once or twice. Yet ubiquitous words are uninformative. How to balance these two conflicting constraints? Tf-Idf weighting!
  - Term Frequency (tf) - For a word w and document D in the corpurs, $count(w, D)$ represents the frequency or raw count of the word in the document. However, we use log-scaling to get $tf(w, D)$.

$$tf(w,D) = \begin{cases} 1 + \log_{10} count(w,D) & if\ count(w,D) > 0 \\ 0 & otherwise \end{cases}$$

  - Inverse Document Frequency (idf) - If in a corpus of N documents, the word w appears in $df(w)$ documents, then

$$idf(w) = \log_{10}\left(\frac{N}{df(w)}\right)$$

- Tf-Idf weighted values replace raw counts in term-document matrix:

$$tfidf_{w,D} = tf(w,D) \times idf(w)$$

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

Tf-Idf weighted term-document matrix for Shakespeare plays. the 0.085 value for *wit* in *As You Like It* is the product of tf = 1 + log10 (20) = 2.301 and idf = .037.
Note that the idf for ubiquitous word good is 0 which makes a word like fool more discriminative through this lens.

Word2Vec

Two issues with previous vector representations of words when dealing with large text corpora:

1. *Long* vectors since for large corpora there will be many documents (≈100K) each forming one dimension in the word vector.
2. Most vectors will be *sparse* since most words will appear in only a handful of documents, leaving a lot of zeros.

We need dense vector representations for words ranging from 50-100 dimensions! : *Less storage, more effective & efficient computationally*.

How to get word vectors?

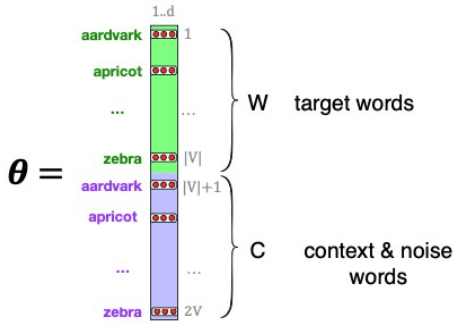
Word2Vec static embeddings (learning one fixed representation of each word) by Mikolov et al.

a. Start with a large corpus of text, fixed vocabulary
b. Self-supervised method, no hand-labeled signals needed.
c. Intuition: instead of counting how often each word *w* occurs near, say, *apricot*, we'll instead train a classifier on a binary prediction task: "Is word *w* likely to show up near *apricot*?"
   We do not care about the prediction but instead take the learned classifier weights as word *embeddings* i.e. vector representations.

Word2Vec : Skip Gram

The intuition of skip-gram is:

1. Treat the target word w and a word in neighboring context window c as positive examples.
2. Randomly sample other words in the lexicon to get negative samples.
3. Use logistic regression to train a classifier to distinguish those two cases.
   Distributional Semantics: Essentially predict the probability $P(+|w,c)$ that c is a context word for w. $P(-|w,c) = 1 - P(+|w,c)$
4. Adjust the learned weights to maximize this probability and use them as the embeddings.



Skip-gram stores two embeddings for each word, one as a target, and one for the word as context. The parameters learnt are two matrices W and C, each containing an embedding for every word.

The classifier:

Imagine a sentence like the following, with a target word apricot, and assume we're using a window of ±2 context words

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
              c1          c2    w     c3         c4
```

We want to train a classifier that, given a tuple (w,c) of a target and a context word, would predict if the candidate context word can appear in the context window of the target word. It should predict true in case of *jam* for *apricot* and false in case of *aardvark*

How does the classifier compute the probability P?

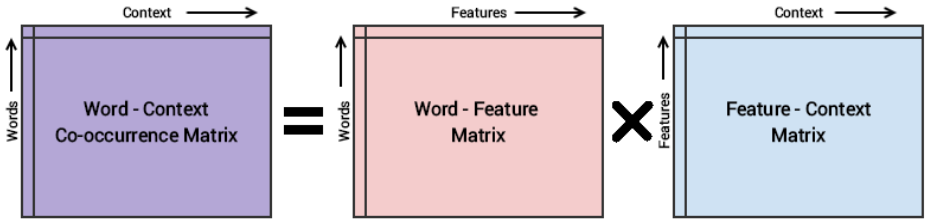
Intuition is to base this probability on the embedding similarity, i.e. the dot product of the embeddings of w and c: $similarity(w,c) \approx c \cdot w$

Turning dot product to probability using the sigmoid transformation $P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+e^{(-c \cdot w)}}$

Independence assumption: Skip-gram makes the simplifying assumption that all context words are independent, allowing us to just multiply their probabilities. So, for a word w and a context window of length L we have :

$P(+|w,c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i \cdot w)$; $\log P(+|w,c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i \cdot w)$

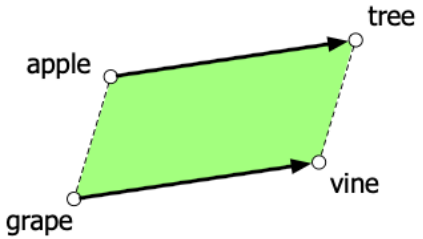
Other Static Embeddings

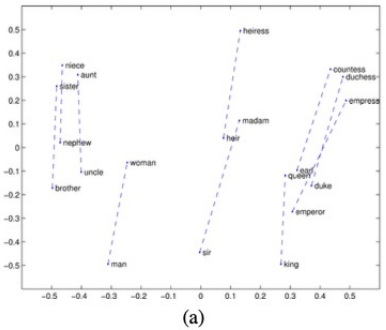


- Fasttext: Addresses few problems with Word2Vec
  - It has no good way to deal with unknown words.
  - A related problem is word sparsity, such as in languages with rich morphology, where some of the many forms for each noun and verb may only occur rarely.
  - Fasttext deals with these problems by using sub-word models, representing each word as itself plus a bag of character n-grams, with special boundary symbols < and > added to each word. For example, with *n* = 3 the word *where* would be represented by the sequence *<where>* plus the character n-grams: *<wh, whe, her, ere, re>*
- GloVe: Short for Global Vectors
  - The model is based on capturing global corpus statistics.
  - GloVe is based on ratios of probabilities from the word-word co-occurrence matrix, combining the intuitions of count-based models like PPMI while also capturing the linear structures used by methods like Word2Vec

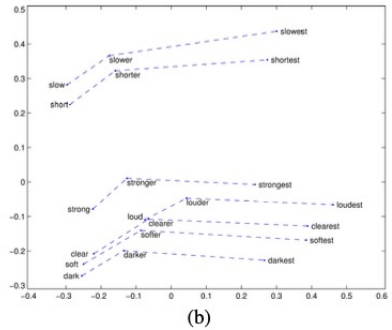
Semantic Properties of Embeddings

- Two words have first-order co-occurrence (sometimes called syntagmatic association) if they are typically nearby each other. Thus, *wrote* is a first-order associate of *book* or *poem*.
- Two words have second-order co-occurrence (sometimes called paradigmatic association) if they have similar neighbors. Thus, *wrote* is a second-order associate of words like *said* or *remarked*.
- **Parallelogram Model** for solving simple analogy problems of the form *a is to b as a* is to what?*. In such problems, a system is given a problem like *apple:tree::grape:? Vine.* Vector from the word $\overrightarrow{apple}$ to the word $\overrightarrow{tree}$ (= $\overrightarrow{tree} - \overrightarrow{apple}$) is added to the vector $\overrightarrow{grape}$;
  the nearest word to the resulting point is returned.
- There are some caveats. For example, the closest value returned by the parallelogram algorithm in word2vec or GloVe embedding spaces is usually not in fact b* but one of the 3 input words or their morphological variants (i.e., *cherry:red :: potato:x* returns *potato* or *potatoes* instead of *brown*), so these must be explicitly excluded.





Relational properties of the GloVe vector space when projected on 2d space



Offsets capture comparative and superlative morphologies

Evaluation

How to evaluate word vectors?

- Extrinsic Evaluation: Generate embedding and use the effectiveness of a specific NLP task that is performed using the generated embeddings. Example: Customer Chatbot
- Intrinsic Evaluation: The most common metric is to test their performance on similarity, computing the correlation between an algorithm's word similarity scores and word similarity ratings assigned by humans. Can also use embeddings to evaluate an intermediate task like POS Tagging

All embedding algorithms suffer from inherent variability.

For example, because of randomness in the initialization and the random negative sampling, algorithms like word2vec may produce different results even from the same dataset, and in- dividual documents in a collection may strongly impact the resulting embeddings.

When embeddings are used to study word associations in particular corpora, therefore, it is best practice to train multiple embeddings with bootstrap sampling over documents and average the results.